Sesion 4: Testing IA-Driven

Playwright · Screenplay · BDD (Python) · MCP — de casos manuales a automatizacion inteligente

Certificacion 5: QA Cloud-Native - Automatizacion Inteligente con IA Generativa

1. Pre-requisitos
2. Recordar los casos
3. Del caso manual a Gherkin
4. Herramientas IA y framework
5. Pipeline de automatizacion (evidencia)
6. Configurar MCP
Ejercicios
Cierre

0. Guia rapida: que hacer para que TODO funcione

Resumen para dummies — lo tocas una vez, lo demas ya esta hecho

⚠️ Todo corre en **Azure** (no en tu PC). Tu solo ajustas **3 datos** y configuras **1 token**. El framework, el pipeline y los scripts ya estan listos.

Lo que necesitas tener (una sola vez)

Que	Donde
Cuenta Azure DevOps + proyecto	dev.azure.com
Repo `ProyectoTienda` en Azure Repos	tu repo
Test Plan con 3 Test Cases (Catalogo, Carrito, Pedidos)	Azure Test Plans
Un **PAT** con scope *Test Management (Read & write)*	Azure → avatar → Personal Access Tokens
Claude Desktop + cuenta (plan Free) — solo para el MCP	claude.com/download

Los 3 datos que SIEMPRE ajustas con TUS valores

1. **IDs de los Test Cases** → en los `.feature`, como tag `@TC<id>` (el ID sale de la columna "Test Case Id" en Test Plans).
2. **IDs del Plan y el Suite** → en `azure-pipelines.yml`, variables `TEST_PLAN_ID` y `TEST_SUITE_ID` (salen de la URL: `...?planId=14&suiteId=18`).
3. **El token (PAT)** → como variable **secreta** del pipeline, llamada `AZURE_DEVOPS_PAT` (NUNCA en el YAML ni en el repo).

✏️ **EJERCICIO PRACTICO**

Crear el PAT correcto (el paso que mas falla)

1 Azure DevOps → **avatar (arriba der.)** → **Personal Access Tokens** → **New Token**.

2 Organization: la tuya. Scopes: clic en **"Show all scopes"** → **Test Management** → **Read & write** (o Full access).

3 **Create** → COPIA el token completo (solo se ve una vez).

4 Pipeline → **Edit** → **Variables** → **New variable**: Name `AZURE_DEVOPS_PAT`, pega el token, marca **Keep this value secret**.

Cada vez que quieras ver resultados

commit + push → Azure corre el pipeline solo → revisas: **(1)** pestaña **Tests** del run (Pass/Fail), **(2)** artefacto **allure-report-html** (abre `index.html` con doble clic: captura por paso, video, trace), **(3) Azure Test Plans** → **Execute** (los casos pasan a Passed/Failed solos).

📘 DOCUMENTACION DE REFERENCIA

Errores tipicos (lo que veras en el log) y su causa

En el log	Significa	Arreglo
`longitud 19 chars`	PAT incompleto	Copia el token completo
`HTTP 401`	al PAT le falta el scope Test Management	Crea un PAT con ese scope (Work Items NO sirve)
`Test plan X not found`	`TEST_PLAN_ID` mal	Copialo de la URL (`planId=`)
`Test Points encontrados: {}`	los casos no estan en ese plan	Verifica plan/suite

💡 En una frase: **una vez** creas el PAT (secreto); **ajustas** los tags `@TC` y las variables del plan; y **cada vez** haces push y Azure hace el resto.

1. Pre-requisitos de la sesion

Lo que debes tener listo en Azure (nada corre en tu PC)

Hoy **todo corre en Azure** (pipeline + Test Plans). No necesitas levantar nada en tu maquina: el pipeline arranca la app, corre las pruebas y publica la evidencia. Esto es lo que debes tener listo en Azure antes de empezar.

Que	Donde	Para que
Repo del Dia 3 (`ProyectoTienda`)	Azure Repos	la app + los tests sencillos; hoy le agregamos la carpeta `e2e/`
Test Plan del Dia 3	Azure Test Plans	los 3 casos manuales que vamos a enlazar con la automatizacion
Pipeline (agente `ubuntu-latest`)	Azure Pipelines	Node y Python ya vienen en el agente; no instalas nada local
PAT (token)	variable secreta del pipeline	para que el pipeline actualice los Test Points (lo vemos en la seccion 3)

La app bajo prueba (ya incluida en el repo)

El repo `ProyectoTienda` trae la **app Tienda** en `prerequisito/app-tienda/` : un front con catalogo, carrito y checkout (todo con atributos `data-testid`) y la API REST, con datos de Faker. Es **la misma app** del Dia 3. Hoy agregamos la carpeta `e2e/` con la automatizacion y un pipeline que, en un solo job, **arranca la app en el agente de Azure** y corre las pruebas contra ella (`BASE_URL=http://localhost:3000` dentro del agente). Nada de esto se levanta en tu PC.

💡 El servidor **MCP de Claude** es lo unico que se usa en una maquina (seccion 6, para crear/reparar pruebas con IA). Su registro lo dejamos para esa seccion, no aqui, porque no hace falta para que el pipeline corra en Azure.

2. Recordar los casos y traducirlos a Gherkin

Tener claros los 3 casos del Dia 3 y convertirlos en escenarios que corren solos

Puente desde el Dia 3 (Azure DevOps)

Ayer, en **Azure Test Plans**, escribimos 3 casos de prueba como pasos manuales. Hoy esos mismos casos se convierten en **pruebas automatizadas** que corren solas, y le sumamos IA para crearlas y repararlas mas rapido. Pero primero hay que tenerlos claros y bien diseñados: ese es el insumo de todo lo que sigue.

Los 3 casos que escribimos ayer en Azure Test Plans

Son el punto de partida de hoy. Cada uno se convertira en un escenario automatizado.

#	Suite	Caso de prueba	Resultado esperado
1	**Catalogo**	Filtrar productos por categoria	Al elegir "Hogar", solo se ven productos de Hogar
2	**Carrito**	Agregar un producto al carrito	El contador del carrito sube a 1
3	**Pedidos**	Confirmar un pedido	Se muestra la confirmacion con numero de pedido

Como se diseña un buen caso (lo que ya aplicamos ayer)

- **Titulo claro:** describe la accion y el objetivo ("Filtrar productos por categoria").
- **Pasos concretos:** cada paso es una accion verificable del usuario.
- **Resultado esperado:** que debe verse al final, sin ambiguedad.
- **Dato de prueba:** con que valores se ejecuta (categoria "Hogar", producto 1, correo del cliente).

Un caso bien diseñado se traduce casi 1:1 a Gherkin. Un caso vago (ej. "probar el carrito") no se puede automatizar sin reescribirlo. Por eso este paso va primero.

💡 Hoy los tres pasan de "pasos escritos a mano" a "escenarios Gherkin que corren solos". El caso de negocio es identico; cambia el formato y quien lo ejecuta.

Del caso manual (Test Plans) al escenario Gherkin

El punto clave: el caso NO cambia; cambia como se escribe y quien lo ejecuta.

⚠️ **Aclaracion importante:** en **Azure Test Plans los casos NO estan en Gherkin**. Alla son pasos manuales con un resultado esperado (lenguaje de tester). Cuando pasamos a **automatizar**, esos casos SI se escriben en **Gherkin** (Given/When/Then). Es una traduccion de formato, no un caso nuevo.

La transicion en 2 pasos

Para cada caso seguimos el mismo recorrido: (1) partimos del caso manual, (2) lo reescribimos como escenario Gherkin. En la **seccion 3** veremos como ese Gherkin se vincula de vuelta al Test Case en Azure (por su ID). Veamoslo con un ejemplo generico de login y luego con nuestros casos de la tienda.

Paso 1 — Caso manual en Test Plans

Asi se ve el caso tal cual lo escribe un tester: pasos y resultado esperado.

Test Case (manual) — ejemplo login

```
Step 1: Iniciar sesion con usuario valido
Expected Result: Se muestra el dashboard
```

Paso 2 — Escenario Gherkin equivalente

El mismo caso, ahora en Given/When/Then, listo para automatizar.

Gherkin equivalente — ejemplo login

```
Given el usuario esta en la pagina de login
When ingresa credenciales validas
Then se muestra el dashboard principal
```

Nuestros 3 casos de la tienda, traducidos

Caso manual (Test Plans, Dia 3)	Escenario Gherkin (Dia 4)

Catalogo — Filtrar por categoria
Step 1: Abrir la home → se ve el catalogo
Step 2: Elegir categoria Hogar → solo productos de Hogar
	`Scenario: Filtrar productos por categoria Hogar`
	`Given que abro la home de la tienda`
	`When filtro el catalogo por la categoria "hogar"`
	`Then solo veo productos de la categoria "hogar"`

Carrito — Agregar un producto
Step 1: Abrir la home
Step 2: Agregar un producto → el contador sube a 1
	`Scenario: Agregar un producto al carrito`
	`Given que abro la home de la tienda`
	`When agrego el producto 1 al carrito`
	`Then el contador del carrito muestra 1`

Pedidos — Confirmar un pedido
Step 1: Con un producto en el carrito
Step 2: Confirmar → se ve la confirmacion
	`Scenario: Confirmar un pedido con un producto`
	`Given que abro la home de la tienda`
	`And agrego el producto 1 al carrito`
	`When abro el carrito y confirmo el pedido con el correo "ana@tienda-qa.com"`
	`Then veo el mensaje de pedido confirmado`

Gherkin en ingles, pasos en espanol

Las **palabras clave** de Gherkin van en ingles (`Feature` , `Scenario` , `Given` , `When` , `Then` , `And`) porque es lo que entiende `pytest-bdd` . El **texto** de cada paso va en espanol, para que el caso se lea como negocio. Asi conservamos el lenguaje del tester hispanohablante y la herramienta funciona.

features/catalogo.feature (el caso 1, tal cual corre)

```
Feature: Filtrar el catalogo por categoria

  @catalogo @smoke @TC20
  Scenario: Filtrar productos por categoria Hogar
    Given que abro la home de la tienda
    When filtro el catalogo por la categoria "hogar"
    Then solo veo productos de la categoria "hogar"
    And la cantidad de productos visibles es mayor que cero
```

3. Enlazar los casos con Azure: token, IDs y el script que actualiza los Test Cases

Lo que hay que configurar para que, al correr el pipeline, cada Test Case quede Passed/Failed solo

La idea en una frase

Queremos que al correr la automatizacion en Azure, cada **Test Case** del plan quede en **verde o rojo solo**. Para eso: (1) etiquetamos cada escenario con el **ID de su Test Case**; (2) el framework escribe ese ID en el resultado; (3) un **script Python** usa la API de Azure para mover el Test Point del plan; y (4) ese script se autentica con un **token (PAT)**.

Donde va cada ID (lo unico que cambia en TU cuenta)

Que ID	Donde lo encuentras	Donde lo colocas
ID del Test Case (ej. 19, 20, 21)	Test Plans, columna *Test Case Id* de cada caso	En el `.feature` , tag `@TC<id>` antes del `Scenario`
ID del Test Plan (ej. 14)	URL de Test Plans: `...?planId=14`	YAML, variable `TEST_PLAN_ID`

Que ID	Donde lo encuentras	Donde lo colocas
ID del Suite (ej. 18)	URL: `...&suiteId=18` (o "Pedidos (ID: 18)")	YAML, variable `TEST_SUITE_ID`

1) Los IDs de los casos van en los .feature (tag @TC)

features/*.feature — el ID real de cada Test Case

```
catalogo.feature:   @catalogo @smoke @TC20     # 20 = "Filtrar por categoria"
carrito.feature:    @carrito  @smoke @TC19     # 19 = "Agregar un producto"
pedidos.feature:    @pedidos  @smoke @TC21     # 21 = "Confirmar un pedido"
```

💡 Un escenario sin Test Case en Azure simplemente NO lleva tag @TC: igual se ejecuta y aparece en la pestaña Tests, pero no intenta mover ningun Test Point.

2) El framework propaga el ID al resultado (conftest.py)

El `conftest.py` (hook `pytest_bdd_before_scenario`) lee el tag `@TC` y lo escribe en el JUnit como propiedad del testcase. Ese es el dato que Azure usa para enlazar.

junit.xml que genera (lo que Azure lee para enlazar)

```
<testcase name="test_filtrar_productos_por_categoria_hogar">
  <properties><property name="test_case_id" value="20"/></properties>
</testcase>
```

3) El token (PAT): por que hace falta

Publicar el JUnit deja el resultado en la **pestaña Tests del run**, pero NO cambia el **Test Point** del plan (pestaña Execute). Para eso el script llama a la **REST API de Azure Test**, y esa API necesita autenticarse con un **Personal Access Token (PAT)** con el scope **Test Management (Read & write)**.

⚠️ **El PAT es una credencial secreta.** NUNCA va en el YAML, ni en el repo, ni en capturas o chats. Va SOLO como **variable secreta** del pipeline. Si se expone, revocalo y crea otro.

crear y guardar el PAT (una vez)

```
1) Azure DevOps -> avatar -> Personal Access Tokens -> New Token
2) Organization: la tuya (ej. oach82)    Expiration: 30 dias
3) Scopes: "Show all scopes" -> Test Management -> Read & write  (o Full access)
4) Create -> COPIA el token completo (solo se ve una vez)
5) Pipeline -> Edit -> Variables -> New variable:
     Name = AZURE_DEVOPS_PAT   Value =    [x] Keep this value secret
```

4) Las variables del plan en el YAML

azure-pipelines.yml (bloque variables) — NO secretos

```
variables:
  TEST_PLAN_ID:  '14'    # ID del Test Plan 'Tienda v1 - Regresion' (de la URL)
  TEST_SUITE_ID: '18'    # un suite del plan (el script recorre todos igual)
  # El PAT NO va aqui: va como variable SECRETA del pipeline (AZURE_DEVOPS_PAT).
```

⚠️ **Truco para no equivocarte con el plan:** el ID del plan NO siempre coincide con lo que esperas (a nosotros "Tienda v1 - Regresion" era el **14**, no el 15). Sacalo de la URL (`planId=`). Si lo pones mal, el script dice `Test plan X not found` y LISTA los planes que si existen con su ID — copias de ahi el correcto.

5) El script Python que actualiza los Test Cases

`e2e/tools/publicar_test_plan.py` hace el cierre: lee el `junit.xml`, saca cada `test_case_id` y su resultado, busca el Test Point en el plan (recorriendo todos los suites) y publica el estado via API. Lo ejecuta el pipeline despues de pytest.

lo que hace el script (resumen)

```
1. Lee junit.xml -> {'20': 'Passed', '19': 'Passed', '21': 'Passed'}
2. Autentica con AZURE_DEVOPS_PAT (scope Test Management)
3. Busca los Test Points del plan 14 (todos los suites: Catalogo, Carrito, Pedidos)
4. Crea un Test Run y publica Passed/Failed en cada punto
5. Cierra el run -> el Test Case queda verde/rojo en la pestana Execute
```

💡 Diagnostico seguro: el script NO imprime el token, solo su *longitud*, y ante error muestra el **codigo HTTP** (401 = scope/permiso del PAT; 404 = plan/ID mal). Si falta el PAT o algun ID, el paso se salta sin romper el pipeline: el resultado igual queda en la pestaña Tests.

Resumen de los 3 lugares

(1) IDs de casos → tags `@TC` en los `.feature`. **(2)** ID de plan y suite → variables `TEST_PLAN_ID`/`TEST_SUITE_ID` del YAML (no secretos). **(3)** El token → variable secreta `AZURE_DEVOPS_PAT`. Con eso, al correr el pipeline, los Test Cases pasan a Passed/Failed solos.

4. Herramientas IA del mercado

El panorama SaaS (comparativa) y por que nosotros construimos con Playwright + MCP

Por que "IA-Driven"

Las pruebas E2E se rompen cuando el front cambia (un boton se mueve, un selector cambia). Mantenerlas cuesta mas que escribirlas. La IA ataca justo eso: **autorreparacion** (ajusta selectores que cambiaron), **generacion** (describes el caso en lenguaje natural y lo genera), **validacion visual** (compara lo que el usuario VE) y **analisis** (agrupa fallos y sugiere causa raiz).

A) El panorama SaaS (solo comparativa)

⚠️ **¿Cual usamos en esta sesion?** Ninguna de las SaaS de abajo. Las cuatro (Testim, Mabl, Functionize, Applitools) se muestran **solo como comparativa** para conocer el mercado. La herramienta de IA que **realmente integramos y corre** es el **servidor MCP de Playwright** (seccion 6). Es gratuito, abierto y vive en nuestro propio framework.

AUTORREPARACION

Testim.io

Localizadores "inteligentes": cuando el DOM cambia, ajusta el selector solo. Reduce el mantenimiento de suites grandes.

ML + VISUAL

Mabl

Machine learning sobre las corridas: detecta regresiones visuales y de rendimiento, y auto-cura pruebas.

INTENT-BASED

Functionize

Describes la intencion en lenguaje natural y genera la prueba. Enfoque "escribe lo que quieres probar".

VISUAL IA

Applitools

Validacion visual con IA (Visual AI): compara lo que el usuario VE, no solo el DOM. Ignora ruido, detecta cambios reales.

Resumen: que se usa y que no

Herramienta	Rol en la sesion	Corre en clase
Testim.io	Comparativa (autorreparacion)	No — solo se menciona
Mabl	Comparativa (ML + visual)	No — solo se menciona
Functionize	Comparativa (intent-based)	No — solo se menciona
Applitools	Comparativa (visual IA)	No — solo se menciona
Playwright MCP	**La que integramos y usamos**	**Si — de verdad**

¿Por que construimos con Playwright + MCP y no compramos una SaaS?

Las SaaS son potentes pero son cajas con costo y lock-in. Con **Playwright + Screenplay + BDD** tienes un framework abierto, versionado en tu repo y que corre en cualquier pipeline. Y con **MCP** le sumas la capa de IA (generar/reparar) sin atarte a un proveedor. Lo mejor de los dos mundos: control + inteligencia.

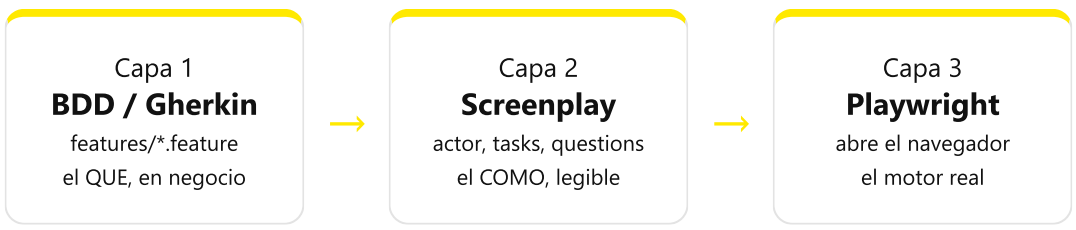
💡 Las cuatro SaaS requieren cuenta/trial y quedan como panorama de mercado. Lo que de verdad instalas y pruebas hoy es el servidor MCP de Playwright, gratuito y abierto.

5. La automatizacion en detalle: Playwright + Screenplay + BDD

Tres capas que hacen las pruebas legibles y mantenibles

B) La decision: framework abierto + IA via MCP

En vez de una SaaS cerrada, construimos con piezas abiertas y le enchufamos IA con MCP. Asi se compone nuestro framework, de arriba hacia abajo:



Patron Screenplay en una frase

En vez de un "page object" lleno de metodos, modelas la prueba como una obra: hay un **actor** (el usuario) que tiene **habilidades** (navegar), ejecuta **tareas** (agregar al carrito) y responde **preguntas** (¿cuanto marca el contador?). El test se lee casi como lenguaje natural.

Pieza Screenplay	Que es	Ejemplo en la tienda
Actor	Quien usa la app	`Ana`
Ability	Lo que PUEDE hacer	`NavegarConElNavegador` (Playwright)
Task	Accion de negocio	`AgregarProductoAlCarrito.con_id(1)`
Interaction	Accion atomica	`HacerClic.en("agregar-1")`
Question	Lo que OBSERVA	`ContadorCarrito.valor()`

Como se lee un paso con Screenplay (tests/steps_comunes.py)

```
@when(parsers.parse("agrego el producto {producto_id:d} al carrito"))
def agregar_producto(actor, producto_id):
    actor.intenta(AgregarProductoAlCarrito.con_id(producto_id))
```

```
@then(parsers.parse("el contador del carrito muestra {esperado:d}"))
def contador_carrito(actor, esperado):
    assert actor.pregunta(ContadorCarrito.valor()) == esperado
```

💡 Fijate que el step no sabe de clicks ni de selectores: solo pide tareas y preguntas al actor. Si el front cambia, tocas una Task o Question, no los 40 escenarios que la usan. Eso es mantenibilidad.

📘 DOCUMENTACION DE REFERENCIA

Las piezas, con sus docs oficiales

- **Playwright for Python:** motor de automatizacion multinavegador (Chromium, Firefox, WebKit). Usa localizadores robustos como `get_by_test_id`, con auto-waiting (espera sola a que el elemento este listo).
- **pytest-bdd:** implementa un subconjunto de Gherkin sobre el runner de pytest. Cada `Given/When/Then` se mapea a una funcion Python; los `.feature` viven aparte del codigo.
- **Patron Screenplay:** patron de diseño centrado en actores (origen en Serenity BDD). Favorece pruebas legibles, por capas y mantenibles frente al Page Object clasico.

Fuentes oficiales: Playwright Python · Locators (get_by_test_id) · pytest-bdd · Screenplay Pattern.
Contenido reformulado para cumplir con las restricciones de licencia.

6. Ejecutar la automatizacion en el pipeline y dejar la evidencia

UN solo repo: el del Dia 3 + una carpeta e2e/ con la automatizacion

Un solo repo: la app, los tests sencillos y el E2E

Para mantenerlo simple y que funcione sin complicaciones, todo vive en **el mismo repo del Dia 3** (`ProyectoTienda`). Solo le **agregamos una carpeta** `e2e/` con la automatizacion. No tocamos el Dia 3; lo ampliamos.

Estructura del repo (lo que agregas esta marcado)

```
ProyectoTienda/
├─ prerequisito/app-tienda/   la app (servidor, datos)        [ya existe]
├─ tienda-azure/              tests sencillos: smoke + carga   [ya existe]
├─ e2e/                       automatizacion E2E               [AGREGAR]
│   ├─ features/  screenplay/  tests/
│   ├─ conftest.py  requirements.txt  pytest.ini
└─ azure-pipelines-e2e.yml    el pipeline de hoy               [AGREGAR]
```

⚠️ **El detalle que todos tropiezan:** en Azure cada **stage/job corre en un agente efimero distinto**. Si arrancas la app en un stage, al terminar ese stage el agente se destruye y **la app muere**. Por eso NO sirve "arrancar la app en un stage y usarla en otro". La solucion: hacer **todo en UN SOLO job** — arrancar la app con & y correr smoke, carga y E2E mientras la app sigue viva en ese mismo agente. Al terminar el job, todo muere junto. Limpio.

El pipeline "todo en un job"

Parte del pipeline del Dia 3 (Build → Smoke → Carga) y lo amplia: en el **mismo job**, tras arrancar la app y pasar smoke y carga, corre la **automatizacion E2E** y publica la evidencia. Como es un solo repo, NO necesita `resources: repositories` ni dos `checkout` .

azure-pipelines-e2e.yml (un repo, un job: app viva todo el tiempo)

```
trigger: { branches: { include: [ main ] } }
pool: { vmImage: 'ubuntu-latest' }

variables:
```

```
  BASE_URL: 'http://localhost:3000'
  APP_DIR:  'prerequisito/app-tienda'   # la app (Dia 3)
  SMOKE_DIR: 'tienda-azure'             # smoke + carga (Dia 3)
  E2E_DIR:  'e2e'                        # la carpeta que AGREGAS

stages:
  - stage: CI_E2E
    jobs:
      - job: todo_en_uno                # <-- UN SOLO job: la app no muere entre pasos
        steps:
          - checkout: self              # un solo repo
          - task: NodeTool@0
            inputs: { versionSpec: '20.x' }
          - task: UsePythonVersion@0
            inputs: { versionSpec: '3.11' }
          # build + ARRANCAR la app (queda viva)
          - script: npm install
            workingDirectory: $(APP_DIR)
          - script: |
              node servidor/server.js &
              for i in $(seq 1 20); do curl -sf http://localhost:3000/api/health && break || sleep 2; done
            workingDirectory: $(APP_DIR)
          # smoke y carga (misma app viva)
          - script: BASE_URL=$(BASE_URL) node tests/smoke-api.js
            workingDirectory: $(SMOKE_DIR)
          - script: BASE_URL=$(BASE_URL) CONCURRENCIA=15 DURACION_S=8 node tests/carga-simple.js
            workingDirectory: $(SMOKE_DIR)
          # automatizacion E2E (misma app viva)
          - script: |
              pip install -r requirements.txt
              python -m playwright install --with-deps chromium
              pytest --junitxml=resultados/junit.xml
            workingDirectory: $(E2E_DIR)
            env: { BASE_URL: $(BASE_URL) }
          # evidencia en el run
          - task: PublishTestResults@2
            condition: always()
            inputs:
              testResultsFormat: 'JUnit'
              testResultsFiles: '$(E2E_DIR)/resultados/junit.xml'
              testRunTitle: 'E2E Tienda - Dia 4'
```

💡 La clave es el **job unico**: todos los pasos comparten agente, asi la app arrancada sigue respondiendo en smoke, carga y E2E. Al final del job, el agente (y la app) se destruyen; no queda nada colgando. Las rutas (`APP_DIR`, `SMOKE_DIR`, `E2E_DIR`) son relativas a la raiz del repo: sin prefijos raros porque es un solo repo.

Evidencia rica: capturas por paso, video y trace con Allure

El JUnit dice *que* paso o fallo, pero no **por donde paso la prueba en la app**. Para eso sumamos **Allure**: el `conftest.py` adjunta, en cada escenario, una **captura por cada paso Gherkin** (Given/When/Then), y al cerrar el escenario el **video** de la sesion y el **trace de Playwright** (navegable paso a paso con red, consola y DOM). Si un paso falla, adjunta ademas el **HTML** de la pagina. Todo eso se arma en un **reporte Allure navegable**.

Pasos finales del YAML: generar y publicar el reporte Allure

```
          # automatizacion E2E -> deja allure-results/ (capturas, video, trace)
          - script: |
              mkdir -p resultados
              pip install -r requirements.txt
              python -m playwright install --with-deps chromium
              pytest --junitxml=resultados/junit.xml   # pytest.ini -> --alluredir=allure-results
            workingDirectory: $(E2E_DIR)
            env: { BASE_URL: $(BASE_URL) }
          # reporte Allure (instala el CLI, necesita Java: ya viene en ubuntu-latest)
          - script: |
              curl -sL -o allure.tgz https://github.com/allure-framework/allure2/releases/download/2.30.0/allure-
2.30.0.tgz
```

```
            tar -zxf allure.tgz
            # normal (varios archivos) + single-file (UN html autocontenido)
            ./allure-2.30.0/bin/allure generate allure-results -o allure-report --clean
            ./allure-2.30.0/bin/allure generate allure-results -o allure-single --clean --single-file
          workingDirectory: $(E2E_DIR)
          condition: always()
        # publicar: JUnit en la pestana Tests + Allure (single + completo) como artefactos
        - task: PublishTestResults@2
          condition: always()
          inputs: { testResultsFormat: 'JUnit', testResultsFiles: '**/junit.xml', searchFolder:
'$(E2E_DIR)/resultados' }
        - task: PublishPipelineArtifact@1
          condition: always()
          inputs: { targetPath: '$(E2E_DIR)/allure-single', artifact: 'allure-report-html' }
```

💡 Al terminar el run: la pestaña **Tests** da el Pass/Fail; el artefacto **allure-report-html** es **un unico `index.html` autocontenido** — lo descargas y lo abres con **doble clic** (sin servidor, todas las capturas van embebidas). El artefacto **evidencia-e2e** trae el video `.webm` y el trace `.zip` (`npx playwright show-trace`).

⚠️ **¿Por que un "single-file"?** El `index.html` de un reporte Allure normal carga sus datos por `fetch`, que el navegador bloquea al abrirlo con `file://` (CORS) — por eso "no abre" al hacer doble clic. La version **single-file** mete todo (datos y capturas en base64) en un solo HTML que si abre directo. Para verlo **sin descargar nada**: instala la extension gratuita *Allure Test Reports* en tu organizacion de Azure DevOps y el reporte queda como una **pestaña del run**; o publicalo en una URL (GitHub Pages / Azure Static Web Apps).

🖍️ **EJERCICIO PRACTICO**

Ejercicio 5: ampliar el repo del Dia 3 para que el pipeline haga TODO

No tocamos el pipeline del Dia 3. Agregamos la automatizacion al mismo repo y creamos un **pipeline nuevo** que apunta a un YAML aparte.

1. En tu repo `ProyectoTienda` (el del Dia 3) crea la carpeta `e2e/` y copia dentro la automatizacion del proyecto del curso (`features/` , `screenplay/` , `tests/` , `conftest.py` , `requirements.txt` , `pytest.ini`).
2. Agrega en la raiz del repo el archivo `azure-pipelines-e2e.yml` (el YAML de arriba). Dale un nombre distinto al del Dia 3 para **no tocar** ese pipeline.
3. **Commit + push** a `main` .
4. Azure DevOps → **Pipelines** → **New pipeline** → tu repo `ProyectoTienda` → **Existing Azure Pipelines YAML file** → `/azure-pipelines-e2e.yml` → **Run**.
5. Al terminar, abre la pestaña **Tests** del run (cada escenario Pass/Fail) y descarga el artefacto **allure-report-html**: es un **unico** `index.html` que abres con **doble clic** y muestra cada paso con su **captura de la app**, el video y el trace. (Para verlo sin descargar: extension *Allure Test Reports* o publicarlo en una URL.)

💡 Asi quedan DOS pipelines en el mismo repo: el del Dia 3 (Build/Smoke/Carga, intacto) y el de hoy (todo en un job + E2E + evidencia). El del Dia 3 no se modifica.

El enlace con los Test Cases ya quedo configurado (seccion 3)

El pipeline no solo publica la evidencia: tambien **actualiza los Test Cases** del plan (Passed/Failed) usando los tags `@TC`, las variables `TEST_PLAN_ID`/`TEST_SUITE_ID` y el `AZURE_DEVOPS_PAT` que configuramos en la **seccion 3**. Aqui solo lo corremos y vemos el resultado. Cadena completa: **historia → Test Case (ID) → .feature (@TC) → pipeline → resultado**.

📘 DOCUMENTACION DE REFERENCIA

Pipeline de automatizacion y evidencia — referencia

- **Un repo, un checkout:** al tener la app, los tests sencillos y el e2e en el mismo repo, el pipeline usa un solo `checkout: self` y rutas relativas simples (sin `resources: repositories`).
- **Un solo job:** los pasos de un job comparten agente y sistema de archivos; por eso la app arrancada con `&` sigue viva para los pasos siguientes. Entre jobs/stages NO se comparte: cada uno es un agente nuevo.
- **PublishTestResults@2:** publica los resultados (JUnit) en la pestaña **Tests** del run y, por la propiedad `test_case_id` (del tag `@TC`), enlaza cada resultado con su Test Case y lo marca Pass/Fail.
- **Artefactos del pipeline:** `PublishPipelineArtifact@1` adjunta el XML/reportes al run para descargarlos.
- **Allure:** el `conftest.py` adjunta una captura por paso Gherkin, el video y el trace de Playwright; el pipeline los convierte en un reporte HTML navegable (artefacto `allure-report`).

Fuentes oficiales (Microsoft Learn): PublishTestResults@2 · checkout (self) · Jobs (agente por job) · PublishTestResults@2.
Contenido reformulado para cumplir con las restricciones de licencia.

⚠️ Hoy el pipeline corre en **un navegador (chromium)**, con la app viva dentro del job. En el **Dia 5** esto evoluciona: matriz multi-navegador, ejecucion en **Sauce Labs** (navegadores reales) y **triaje de fallos con IA**. Mismo pipeline, mas potencia.

7. Configurar el MCP con Claude: la IA con manos

Claude Desktop + servidor MCP de Playwright: crear/reparar pruebas con IA (y como encaja con Azure)

Que es MCP y por que con Claude

MCP (Model Context Protocol) es un estandar abierto que conecta un asistente de IA con herramientas externas. El **servidor MCP de Playwright** le da al agente "manos" sobre un navegador real: navegar, hacer clic, tomar un *snapshot de accesibilidad* de la pagina y proponer codigo. En esta sesion usamos **Claude (Claude Desktop)** como el "cerebro": es el cliente MCP que conversa contigo y manda ordenes al servidor de Playwright. El servidor solo ejecuta las acciones del navegador; Claude decide que hacer.

⚠️

¿Como encaja el MCP si TODO corre en Azure?

Esta es la duda clave. El **MCP NO corre en el pipeline de Azure**. Corre en **tu maquina**, con Claude Desktop, y es una herramienta de **autoria**: te ayuda a **crear y reparar** los escenarios ANTES de subirlos. El flujo es: **1)** En tu PC, Claude + MCP exploran la app y te proponen/arreglan un escenario Gherkin y sus selectores. **2)** Tu revisas, lo guardas en `features/` y `tests/`, y haces **commit + push**. **3)** A partir de ahi, es **Azure** quien ejecuta esas pruebas en el pipeline (sin MCP, sin Claude). La IA ayuda a ESCRIBIR; Azure se encarga de EJECUTAR.



💡 Por eso el MCP es lo unico que se toca en una maquina. Para que Claude pueda "ver" la app durante la autoria, apunta el MCP a la **app desplegada** (la URL del Dia 3) o levanta la app un momento solo mientras trabajas con Claude. No es parte del pipeline.

Snapshot de accesibilidad, no capturas

En vez de mandarle una imagen al modelo, el servidor le da a Claude un **arbol de accesibilidad** estructurado de la pagina (roles, nombres, estados). Es mas preciso y barato en tokens que una captura, y permite razonar sobre selectores estables como `data-testid`.

Pre-requisitos para esta parte

- **Cuenta de Anthropic (Claude):** gratuita; el plan Free alcanza para la demo.
- **Claude Desktop:** la app de escritorio (Windows/macOS/Linux). El MCP local se configura ahi, no en la version web.
- **Node.js** instalado: npx descarga el servidor de Playwright la primera vez (no instalas nada mas).
- La **app accesible** para que Claude la "vea": usa la **URL desplegada** (Dia 3) o levanta la app un momento solo durante esta parte. Esto es solo para la autoria con IA, no para el pipeline.

🖍️ EJERCICIO PRACTICO

Paso A: crear la cuenta e instalar Claude Desktop

1. **Crear cuenta:** entra a `claude.ai` → **Sign up**. Registrate con tu correo (o con Google/Apple), verifica el email y completa la verificacion por telefono. Quedas en el chat de Claude.
2. **Descargar la app:** ve a `claude.com/download` y descarga **Claude Desktop** para tu sistema (en Windows, el instalador .exe/MSIX).
3. **Instalar e iniciar sesion:** ejecuta el instalador, abre Claude Desktop e inicia sesion con la cuenta Anthropic que creaste.

💡 El **chat de Claude es gratuito** y suficiente para esta demo. Hay planes de pago (desde ~17 USD/mes) para funciones avanzadas, pero no son necesarios aqui.

🖍️ EJERCICIO PRACTICO

Paso B: anadir el servidor MCP de Playwright a Claude Desktop

1. En Claude Desktop abre **Settings** (menu del sistema / icono) → pestaña **Developer** → boton **Edit Config**. Eso abre el archivo `claude_desktop_config.json` .
2. Pega el bloque de abajo. Ojo: la clave en Claude es `mcpServers` (no `servers`). Si el archivo ya tiene contenido, agrega solo la entrada `playwright` dentro de `mcpServers` .
3. Guarda el archivo y **cierra Claude Desktop por completo** (que no quede en la bandeja). Vuelve a abrirlo: solo asi recarga la config.
4. **Verifica:** en una conversacion nueva, el icono de herramientas (o Search and tools) debe listar las acciones de **playwright** (navegar, click, snapshot...). Si aparecen, el servidor quedo conectado.

claude_desktop_config.json — Settings → Developer → Edit Config

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    }
  }
}
```

Donde esta el archivo (por si quieres abrirlo a mano)

```
Windows : %APPDATA%\Claude\claude_desktop_config.json
macOS   : ~/Library/Application Support/Claude/claude_desktop_config.json
```

⚠️ En Windows, npx debe estar en el PATH (viene con Node.js). Si Claude no lista las herramientas, revisa que Node este instalado (`node -v`), que la clave sea `mcpServers` y que cerraste y abriste Claude por completo (no solo la ventana).

🖍️ EJERCICIO PRACTICO

Paso C: usar Claude + Playwright para crear/reparar una prueba

1. Con la app accesible (URL desplegada o levantada un momento), escribele a Claude en lenguaje natural. La primera vez te pedira permiso para usar la herramienta: acepta.

2. **Lleva el resultado al framework:** el escenario que Claude proponga va a `features/` y los pasos nuevos a `tests/steps_comunes.py`, reutilizando las tasks/questions del patron Screenplay.

Prompts para Claude (apunta a tu URL: la desplegada o la que levantaste)

- "Abre `<URL-de-la-app>`, filtra por categoria Hogar y dime cuantos productos quedan."
- "Genera un escenario Gherkin (keywords en ingles, texto en espanol) para agregar dos productos distintos y confirmarlos."
- "Este selector fallo tras un cambio de UI; proponme uno estable basado en `data-testid`."
- "Lee `features/carrito.feature` y sugiere un caso de borde que falte."

🖍️ **EJERCICIO PRACTICO**

Ejercicio 7: del MCP al pipeline (el flujo completo)

Cierra el circulo: lo que la IA te ayuda a crear, lo ejecuta Azure.

1. En Claude (con MCP), pide: "Genera un escenario Gherkin para **buscar un producto por nombre** y verificar que aparece". Revisa lo que propone.

2. Pega el escenario en un nuevo `features/busqueda.feature` y, si hace falta, agrega el step en `tests/steps_comunes.py` reutilizando las tasks/questions existentes.

3. Si el caso tiene un Test Case en Azure, ponle su tag `@TC<id>` (como en la seccion 3). Si no, dejalo sin tag.

4. **Commit + push.** El pipeline corre solo: ejecuta tu nuevo escenario en Azure, publica la evidencia (Allure) y, si tiene tag, actualiza el Test Case. **Tu no ejecutas nada local.**

💡 Este ejercicio deja clara la division de trabajo: **MCP = autoria asistida por IA (en tu PC)**; **Azure = ejecucion y evidencia (en el pipeline)**. La IA no corre en el pipeline; su valor es que escribas y repares pruebas mas rapido antes de subirlas.

⚠️ La IA **asiste**: Claude genera y repara, pero tu revisas y versionas. En Azure las pruebas corren con `pytest` dentro del pipeline. MCP no reemplaza el framework ni el pipeline; acelera la escritura.

📘 DOCUMENTACION DE REFERENCIA

Claude Desktop + MCP de Playwright — referencia

- **Cuenta:** se crea en `claude.ai` (Sign up). El plan Free sirve para la demo.
- **App de escritorio:** se descarga de `claude.com/download`; el MCP local vive en Claude Desktop, no en la web.
- **Configuracion:** Settings → Developer → Edit Config edita `claude_desktop_config.json` con la clave `mcpServers`. Requiere **reiniciar Claude por completo**.
- **Servidor:** `@playwright/mcp` se ejecuta con `npx`; no requiere instalacion previa.

Fuentes: Descargar Claude Desktop · Instalar Claude Desktop · Playwright MCP en Claude Desktop · microsoft/playwright-mcp.
Contenido reformulado para cumplir con las restricciones de licencia.

Ejercicios: automatizar los 3 casos

Correr en el pipeline, leer el framework y revisar la evidencia

✏️ EJERCICIO PRACTICO

Ejercicio 1: correr los 3 casos en el pipeline (Azure)

1. Haz **commit + push** a `main`. El pipeline arranca solo: levanta la app en el agente, corre los 4 escenarios y publica la evidencia.
2. Abre el run → pestaña **Tests**: veras los 4 escenarios en verde (los 3 casos + el extra de carrito), cada uno con su `test_case_id`.
3. En **Azure Test Plans**, pestaña Execute: tus Test Cases 19/20/21 pasaron de Active a **Passed**.

Pestaña Tests del run — resultado esperado

```
test_agregar_un_producto_al_carrito              Passed
test_agregar_dos_productos_distintos_al_carrito  Passed
test_filtrar_productos_por_categoria_hogar       Passed
test_confirmar_un_pedido_con_un_producto         Passed
4 passed
```

✏️ EJERCICIO PRACTICO

Ejercicio 2: leer el flujo completo de un caso

1. Abre `features/pedidos.feature`: el caso en Gherkin (negocio).
2. Abre `tests/steps_comunes.py`: cada `Given/When/Then` pide tareas al actor.
3. Abre `screenplay/tasks.py` y `questions.py`: ahi vive el "como". Sigue la cadena: feature → step → task → interaction → Playwright.

✏️ EJERCICIO PRACTICO

Ejercicio 3: revisar la evidencia Allure del run

1. En el run, descarga el artefacto **allure-report-html** y abre su `index.html` con doble clic.
2. Entra a un escenario y revisa la **captura por cada paso** (Given/When/Then): la evidencia visual de por donde paso la prueba en la app.
3. Abre el artefacto **evidencia-e2e** y mira el `video.webm` y el `trace.zip` (`npx playwright show-trace`).

💡 Todo corre en Azure: no ejecutas nada en tu PC. La misma portabilidad (una URL, un pipeline) es la base de la Sesion 5, donde lo llevamos a **Sauce Labs** (navegadores reales en la nube).

A tener en cuenta y puente al Dia 5

Lo que construiste y hacia donde va

Lo que construiste hoy

- Un **framework E2E real**: Playwright + Screenplay + BDD en Python, con Faker.
- Los **3 casos del Dia 3** traducidos de Test Plans a Gherkin y automatizados (4 escenarios en verde).
- Un **pipeline de automatizacion** (mismo repo, carpeta `e2e/`, todo en un job) que corre pytest y **publica la evidencia** (JUnit) en el propio pipeline de Azure.
- La integracion de **MCP con Claude**: Claude Desktop conectado al servidor de Playwright, un agente de IA que ayuda a crear y reparar pruebas.

Capa	Herramienta	Corre de verdad
E2E + BDD	Playwright + pytest-bdd (Screenplay)	Si
Datos	Faker (semilla fija)	Si
CI/CD (automatizacion)	Azure Pipelines (app + E2E en un job, publica JUnit + Allure + Test Plan)	Si (todo en Azure)
IA / mantenimiento	Claude Desktop + MCP (servidor Playwright)	Si (cuenta Free de Claude)
SaaS IA (comparativa)	Testim, Mabl, Functionize, Applitools	Demo guiada

Puente al Dia 5

Manana llevamos este framework al **CI/CD con IA** y lo ejecutamos en **Sauce Labs**: navegadores y dispositivos reales en la nube. Las mismas pruebas que hoy corren en el pipeline de Azure correran alla cambiando poco mas que la configuracion, gracias a que ya son portables por `BASE_URL`.

Documentacion oficial por tema:

Playwright Python | pytest-bdd | Screenplay | Claude Desktop | Playwright MCP en Claude | MCP